



main ▾

Modul-Teknologi-Komputasi-Awan / load-balancing-cloudsim /



dzakyahnaf docs: update 5.3 Menjankan Simulasi

194c5de · 4 days ago



Name	Name	Last commit date
..		
img	Refactor code structure for im...	5 days ago
README.md	docs: update 5.3 Menjankan Si...	4 days ago
locustfile.py	refactor structure and add ngi...	5 days ago

README.md



Modul 5 - Cloudsim & Load balancing

Daftar Isi

1. [Pendahuluan](#)
2. [Konsep Load Balancing](#)
3. [Tools yang Digunakan](#)
4. [Implementasi Load Balancing Sederhana](#)
5. [Implementasi Task Scheduling dengan CloudSim](#)

1. Pendahuluan

Dalam sistem berbasis komputasi awan, aplikasi tidak hanya berjalan di satu server, tetapi tersebar di banyak node, virtual machine, atau container yang bekerja secara bersamaan. Setiap permintaan dari pengguna perlu diarahkan ke salah satu dari sekian banyak resource yang tersedia

Semakin tinggi jumlah pengguna dan permintaan, maka muncul tantangan utama:

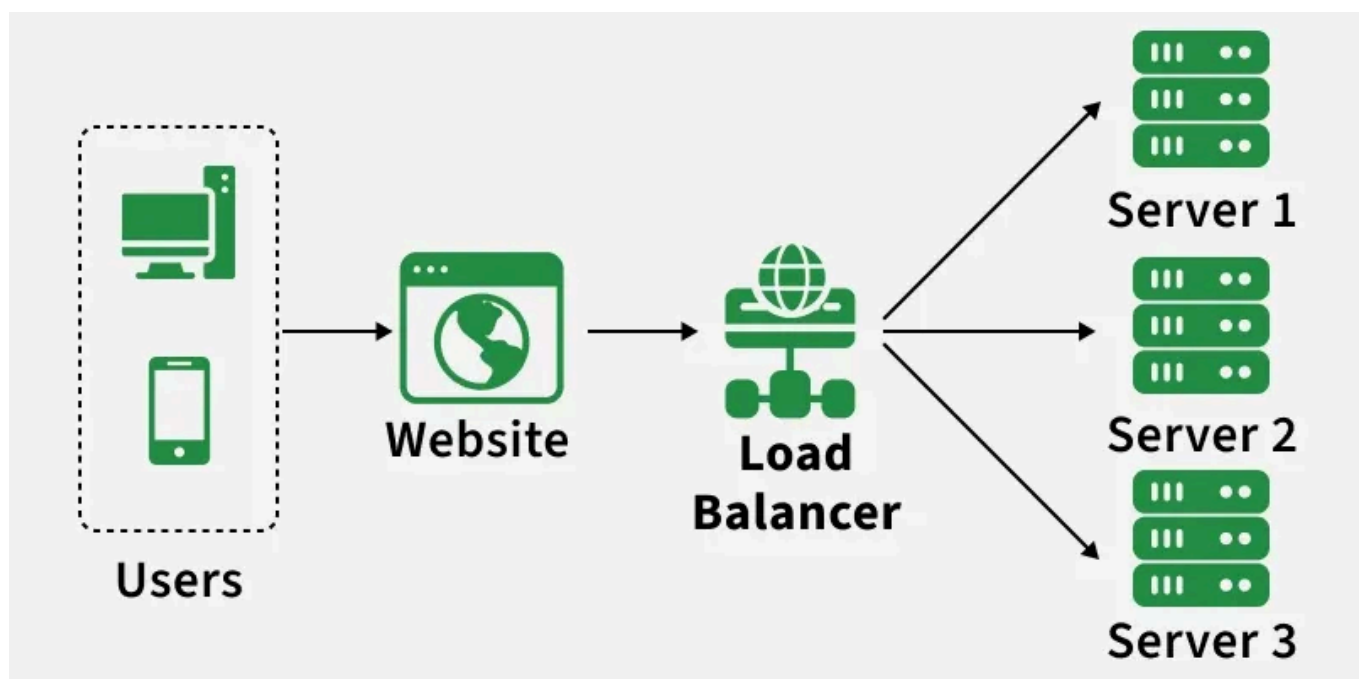
1. Bagaimana cara kita menjaga ketersediaan layanan ketika salah satu server mengalami kegagalan?
2. Bagaimana kita memastikan tidak ada satu server pun yang kelebihan beban sementara server lain menganggur?
3. Bagaimana kita dapat mendistribusikan beban kerja secara efisien agar performa sistem tetap optimal?

Untuk menjawab tantangan tersebut, digunakanlah sebuah konsep yang disebut dengan **load balancing**

2. Konsep Load Balancing

2.1 Apa itu Load Balancing?

Load balancing sendiri adalah sebuah mekanisme yang mengatur distribusi traffic atau beban komputasi ke beberapa server secara merata dan cerdas

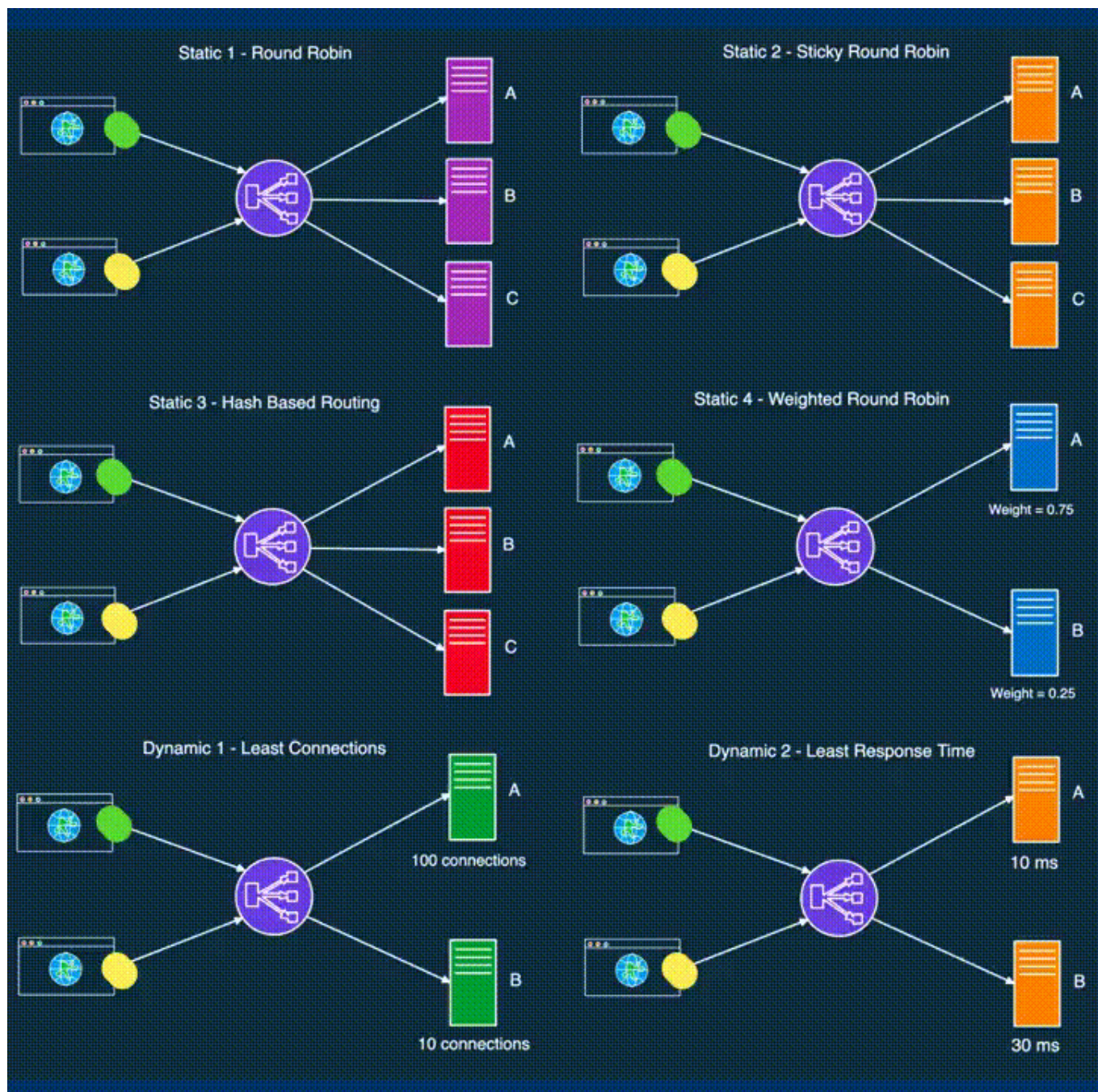


Contoh Penyedia Layanan Cloud Load Balancing:

- Amazon Web Services (AWS) menggunakan Elastic Load Balancing (ELB)
- Google Cloud Platform (GCP) menggunakan Google Cloud Load Balancing
- Microsoft Azure menggunakan Azure Load Balancer

2.2 Load Balancing Algorithms

Sama seperti berbagai sistem di dunia IT, load balancing juga memiliki beberapa algoritma yang berbeda. Setiap algoritma memiliki logika dan cara kerjanya masing-masing dalam menentukan server mana yang paling tepat untuk menerima traffic atau beban kerja pada saat itu



Load balancing sendiri bisa dibagi menjadi 2, yaitu:

1. Static

- **Round robin** Pembagian menggunakan DNS dalam bentuk rotasi
- **Weighted round robin** Pembagian berdasarkan beban yang ditentukan. Jika bisa handle traffic besar, maka weight nya makin besar
- **IP hash** Menggunakan fungsi matematika untuk mengubah IP Address ke hash. Berdasarkan hash tersebut, koneksi dihubungkan pada server tersebut

2. Dynamic

- **Least connection** Melihat server yang memiliki koneksi yang sedikit (Asumsi seluruh kekuatan proses sama)
- **Weighted response time** Melihat rata-rata waktu respons tiap server, tetapi juga melihat jumlah koneksi pada server
- **Resource-based** Melihat sumber daya (eg. CPU) pada server. Memerlukan 'agent' yang dapat memonitor sumber daya server

Dan masih banyak lagi!

3. Tools yang Digunakan

Pada modul kali ini, kita akan melakukan simulasi load balancing dari dua POV yang berbeda menggunakan dua buah tools:

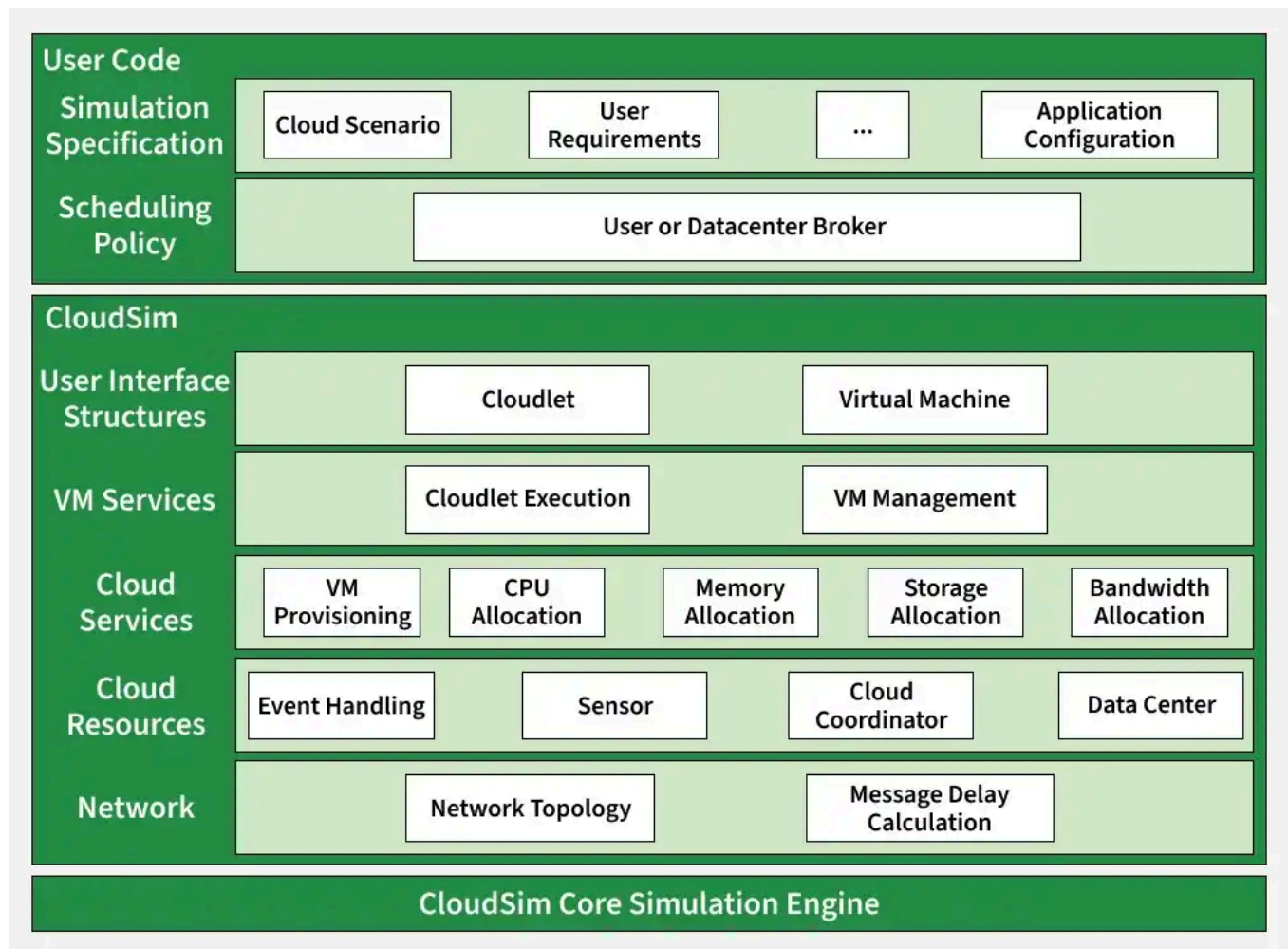
Tool	Fokus	Konsep Utama
NGINX	Jaringan (Network/Web)	Mendistribusikan <i>traffic</i> HTTP/API ke beberapa <i>container</i> aplikasi
CloudSim	Komputasi (Compute/Task)	Mendistribusikan tugas komputasi (<i>Task Scheduling</i>) ke dalam Mesin Virtual (VM)

3.1 NGINX

NGINX adalah program *open-source* berkinerja tinggi yang sering digunakan untuk *web serving*, *reverse proxying*, *caching*, dan **load balancing**. Pada praktikum ini, NGINX akan bertindak sebagai pintu masuk utama yang membagi *request* API dari *client* menuju tiga aplikasi *backend* yang berbeda (menggunakan *Round Robin*)

3.2 CloudSim

CloudSim adalah kerangka kerja atau *framework* berbasis Java yang bersifat *open-source* juga, biasanya digunakan untuk memodelkan dan mensimulasikan infrastruktur serta layanan komputasi awan. Berbeda dengan NGINX yang membagi koneksi web, CloudSim mensimulasikan bagaimana **tugas-tugas komputasi (*Task Scheduling*)** dibagi ke dalam pusat data



Komponen Utama CloudSim:

- **Datacenter:** Memodelkan perangkat keras fisik (*Host/Server*) yang membentuk lingkungan *cloud*. Mengatur kebijakan alokasi VM
- **Broker:** Entitas yang bertindak atas nama pengguna. Bertanggung jawab mengelola pengiriman tugas (*Cloudlet*) ke VM
- **Cloudlet:** Merepresentasikan tugas/aplikasi yang akan dieksekusi (contoh: pemrosesan data). Memiliki parameter seperti ukuran, panjang instruksi (dalam *Million Instructions / MI*), dll
- **VM (Virtual Machine):** Merepresentasikan mesin virtual yang memproses *Cloudlet*. Memiliki atribut seperti RAM, Bandwidth, dan MIPS (*Million Instructions Per Second*)

4. Implementasi Load Balancer Sederhana

Pada modul ini, kita akan mencoba membangun *load balancer* dengan menggunakan **Docker**, **FastAPI** (Python), dan **MongoDB**

Jadi, mari menginstall ketiga library tersebut dengan:

1. Docker dan Docker Compose

Sudah dijelaskan di modul sebelumnya ya ;)

2. FastAPI

```
pip install fastapi
```



Kemudian menginstall `uvicorn` dengan

```
pip install uvicorn
```



3. MongoDB Untuk library ini, hanya akan menginstall `pydantic` untuk keperluan input database dan `pymongo` untuk mengkoneksikan FastAPI dengan database.

```
pip install pymongo pydantic
```



4.1 Menyiapkan Struktur Folder/File

Silakan mengikuti struktur berikut

```
.
├── Awan/
│   ├── app/
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   └── requirements.txt
│   ├── app2/
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   └── requirements.txt
│   └── app3/
│       ├── Dockerfile
│       ├── main.py
│       └── requirements.txt
```



```
├─ nginx/
│   └─ Dockerfile
│       └─ nginx.conf
├─ docker-compose.yml
└─ locustfile.py
```

4.2 Konfigurasi Aplikasi (FastAPI)

Di dalam folder app, app2, dan app3, buat file `main.py` berikut

Note: Jangan lupa diubah yh "This is server A" menjadi B dan C untuk masing-masing folder agar terlihat perbedaannya saat di-test

```
from fastapi import FastAPI, Body, Request
from fastapi.encoders import jsonable_encoder
import pymongo
from pydantic import BaseModel
from bson.objectid import ObjectId
import socket
import time

MONGO_DETAILS = "mongodb://admin:admin@mongodb:27017/"
client = pymongo.MongoClient(MONGO_DETAILS)
db = client['tes'] # Sesuaikan dengan nama database nantinya
collection = db['tes'] # Sesuaikan dengan nama collection nantinya

class Item(BaseModel):
    name: str
    age: int
    rank: str

def myData(data):
    return {
        "id": str(data["_id"]),
        "name": data["name"],
        "age": data["age"],
        "rank": data["rank"],
    }

def myFullData(datas):
    return [myData(data) for data in datas]

def ResponseModel(data, message = "Success"):
    return {"data": [data], "code": 200, "message": message}

def ErrorResponseModel(error, code, message):
    return {"error": error, "code": code, "message": message}
```



```

app = FastAPI()

@app.get('/')
async def home():
    return {"message": "This is server A", "hostname": socket.gethostname()}

@app.get('/fast')
async def hello():
    time.sleep(0.5)
    return {"message": "Hello world from server A", "opt": "fast"}

@app.get('/slow')
async def hello():
    time.sleep(1)
    return {"message": "Hello world from server A", "opt": "slow"}

@app.get("/all")
async def get_all_data():
    return ResponseModel(myFullData(collection.find()), "All Good")

@app.post("/create", response_model=Item)
async def create_data(request: Request, lister: Item = Body(..., embed=True)):
    lister = jsonable_encoder(lister)
    new_data = client.tes.tes.insert_one(lister)
    return lister

@app.get("/get/{id}")
async def get_data(id: str):
    Objinstance = ObjectId(id)
    data = collection.find_one({"_id": Objinstance})
    if data:
        return ResponseModel(myData(data), "A")
    else:
        return ErrorResponseModel("An error occurred.", 404, "Data doesn't exist.")

```

4.3 Konfigurasi Dockerfile

Adapun isi docker diisi berikut:

```

FROM python:3.9-slim
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```



Docker akan membuat folder app dan akan menginstall seluruh requirement di dalam file `requirement.txt` . Saat docker diluncurkan, Docker python akan menjalankan `uvicorn` pada port `8000` , `uvicorn` akan menjalankan file main dengan aplikasi app (akan dijelaskan pada pembuatan FastAPI)

4.4 Konfigurasi `requirement.txt`

Silakan memasukkan library-library yang akan digunakan:

```
fastapi==0.78.0
uvicorn==0.18.2
pymongo
pydantic
uuid
```



4.5 Konfigurasi `nginx`

Pada `nginx`, kita akan mengkonfigurasi `Dockerfile` dan file konfigurasi.

```
FROM nginx
RUN rm /etc/nginx/conf.d/default.conf
COPY ./nginx.conf /etc/nginx/conf.d/default.conf
```



Dengan file konfigurasi `nginx.conf` sebagai berikut:

```
upstream app {
    server app:8000;
    server app2:8000;
    server app3:8000;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://app;
    }
}
```



Mengingat seluruh app akan dijalankan di docker port **8000** dan `nginx` akan dijalankan pada host port **80**. `proxy_pass http://app` nama app menyesuaikan nama `uvicorn` yang dijalankan. Dari upstream yang kita masukkan, load balancer yang akan digunakan adalah **Round-Robin**

4.6 Konfigurasi docker-compose

Pada docker compose, ada beberapa docker image yang akan digunakan, berupa:

- NGINX
- Mongo dan Mongo-Express

Buatlah docker-compose dengan isi seperti di bawah ini:

```
services:
  app:
    build: ./app
    ports:
      - "8001:8000"
    depends_on:
      - mongodb
  app2:
    build: ./app2
    ports:
      - "8002:8000"
    depends_on:
      - mongodb
  app3:
    build: ./app3
    ports:
      - "8003:8000"
    depends_on:
      - mongodb
  nginx:
    build: ./nginx
    ports:
      - "80:80"
      - "443:443"
    depends_on:
      - app
      - app2
      - app3
  mongodb:
    image: mongo
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=admin
    volumes:
      - mongodb_data:/data/db
  mongo-express:
    image: mongo-express
```



```
ports:
  - "8082:8081"
environment:
  - ME_CONFIG_MONGODB_ADMINUSERNAME=admin
  - ME_CONFIG_MONGODB_ADMINPASSWORD=admin
  - ME_CONFIG_MONGODB_SERVER=mongodb
  - ME_CONFIG_MONGODB_ENABLE_ADMIN=true
  - ME_CONFIG_BASICAUTH_USERNAME=admin
  - ME_CONFIG_BASICAUTH_PASSWORD=admin
depends_on:
  - mongodb
volumes:
  mongodb_data:
    driver: local
```

Pada service `app` `app2` `app3` , pastikan arah file sudah menuju folder yang memiliki Dockerfile yang telah dibuat sebelumnya.

Adapun host ports yang akan digunakan tiap service adalah **8001 8002 8003** dan docker port seluruhnya diarahkan ke **8000** *Why?* karena kita menjalankan `uvicorn` tiap app di port 8000 namun tiap docker harus dijalankan di port host yang berbeda.

Karena aplikasi akan menghubungkan database, dan database perlu di load terlebih dahulu, maka tambahkan `mongodb`

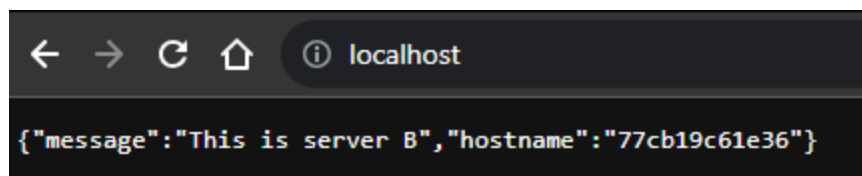
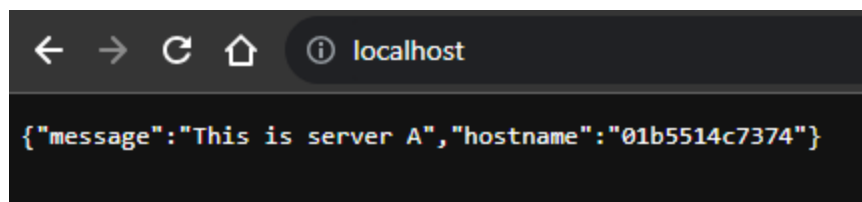
Done 🎉🎉

Apabila sudah sesuai, jalankan docker compose dengan

```
docker-compose up --build
```

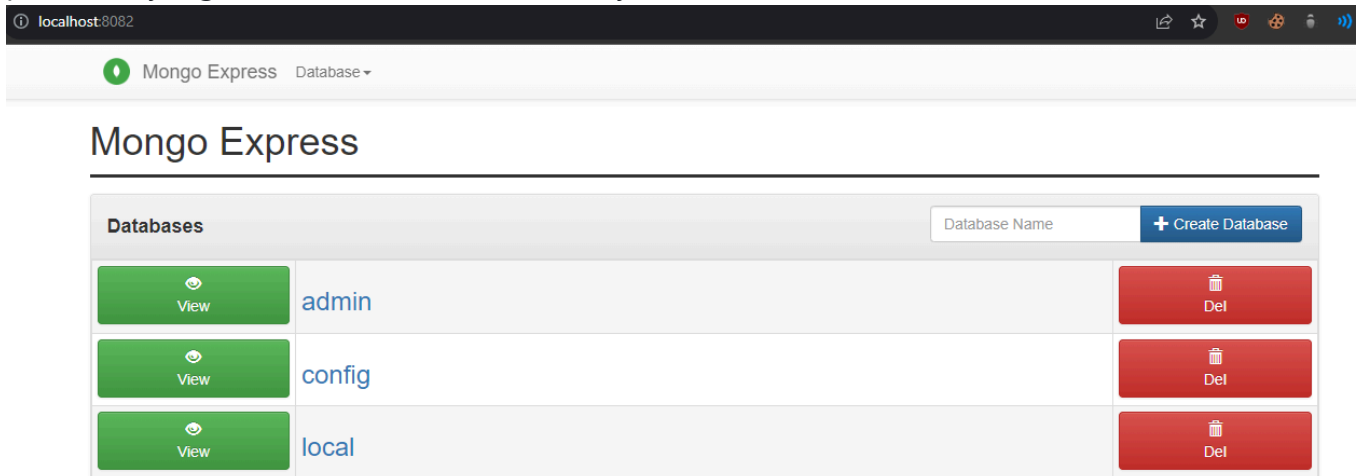


Kemudian buka `localhost` dan coba refresh beberapa kali

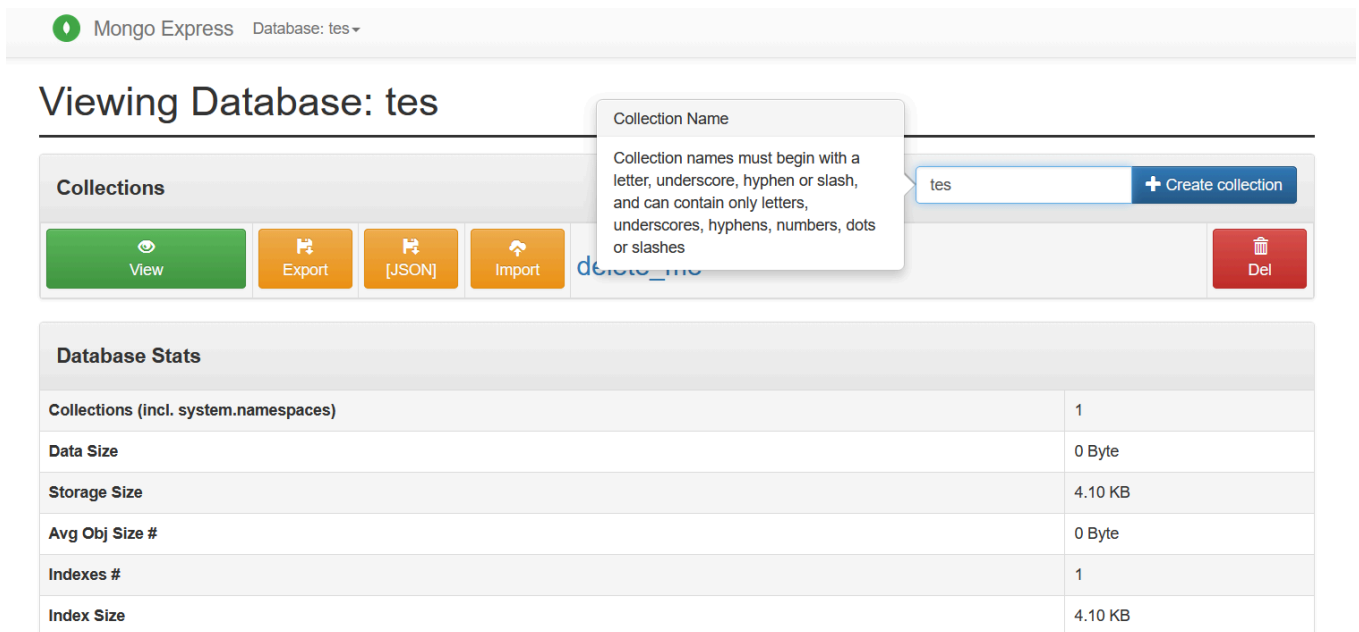


```
localhost  
{ "message": "This is server C", "hostname": "1f4f3764bfd9" }
```

untuk setup mongodb, maka buka localhost:8082 , kemudian masukkan username dan password yang sudah ditentukan sebelumnya.



Langkah selanjutnya: masukkan nama database baru > Tekan create database > Buka database > masukkan nama koleksi baru > tekan Create collection



Pada halaman koleksi, mari buat dokumen baru dengan struktur seperti berikut:

```
{  
  "_id": ObjectId(),  
  "name": "Lorem ipsum",  
}
```



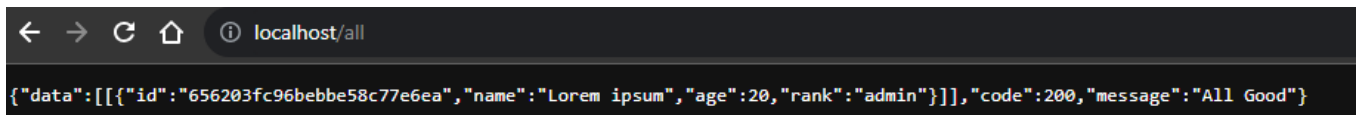
```
    "age": 20,  
    "rank": "admin"  
}
```

Sesuaikan bagian `main.py` di bawah ini sesuai dengan nama databasenya dan collectionnya

```
db = client['tes'] # Ganti 'tes' dengan nama database kalian  
collection = db['tes'] # Ganti 'tes' dengan nama collection kalian
```



Kalian juga bisa buat nama database dan collection nya pake `tes` aja juga gapapa, biar gampang 😊



5. Implementasi Task Scheduling dengan CloudSim

Task Scheduling merupakan proses mengelola eksekusi tugas di dalam cloud. "Tugas" disini adalah komputasi berat seperti pemrosesan data, analisis, dan komputasi paralel.

5.1 Task Scheduling Algorithms

Beberapa algorithm yang akan dibahas pada modul ini:

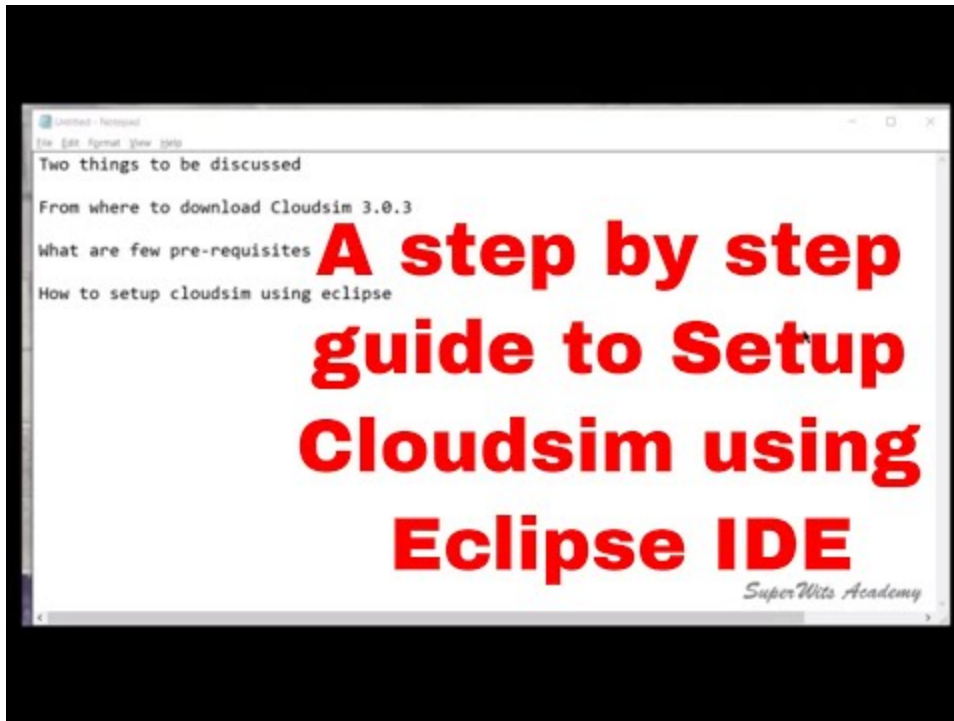
1. **Round Robin:** Task (Cloudlet) diberikan ke mesin (VM) secara melingkar. Mampu menangani banyak pekerjaan secara adil, tetapi bisa menyebabkan waktu tunggu (waiting time) yang lama pada beberapa task besar karena throughput yang kecil
2. **Shortest Job First (SJF):** Memprioritaskan task dengan estimasi waktu proses terkecil. Waktu turnaround cepat, tetapi memerlukan estimasi waktu proses yang sangat akurat agar efektif.

5.2 Instalasi & Persiapan Tools

Beberapa komponen yang perlu diinstall (PALING UPDATE):

- STEP 1: [Eclipse IDE](#) --> [Youtube](#)
- STEP 2: [cloudsim-4](#) --> [Youtube](#)
- [common-math 3.6.1](#)

Video Tutorial menyiapkan tools (Optional):



5.3 Menjalankan Simulasi

Step 1: Buat Java Project Baru di Eclipse

- Buka Eclipse IDE
- File > New > Java Project
- Beri nama project, misalnya **CloudSimScheduling**
- Klik Finish

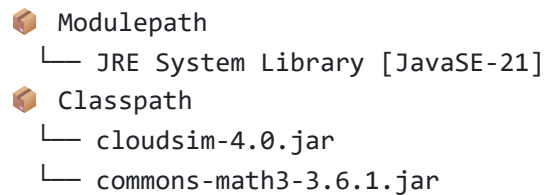
⚠️ Jika Eclipse membuat file `module-info.java`, **hapus file tersebut** (Klik kanan > Delete). File ini membuat project menjadi modular dan akan menyebabkan error karena CloudSim bukan library modular.

Step 2: Tambahkan CloudSim JAR ke Build Path

- Klik kanan project **CloudSimScheduling** > Build Path > Configure Build Path
- Pilih tab **Libraries**
- Klik pada **Classpath** (pastikan BUKAN Modulepath!)
- Klik Add External JARs...
- Browse ke folder `jars/` di dalam folder CloudSim yang telah di-extract, lalu pilih **cloudsim-4.0.jar**
- Tambahkan juga **commons-math3-3.6.1.jar** yang telah didownload sebelumnya

- Klik Apply and Close

Hasil akhir Build Path seharusnya terlihat seperti ini:

A screenshot of the Eclipse IDE's 'Build Path' dialog box. The 'Libraries' tab is selected, showing a tree structure. Under 'Modulepath', there is 'JRE System Library [JavaSE-21]'. Under 'Classpath', there are 'cloudsim-4.0.jar' and 'commons-math3-3.6.1.jar'. A copy icon is visible in the top right corner of the dialog.

```
Modulepath
└─ JRE System Library [JavaSE-21]
Classpath
└─ cloudsim-4.0.jar
└─ commons-math3-3.6.1.jar
```

Step 3: Download Repository Task Scheduling

Download repository berikut sebagai ZIP, lalu extract:

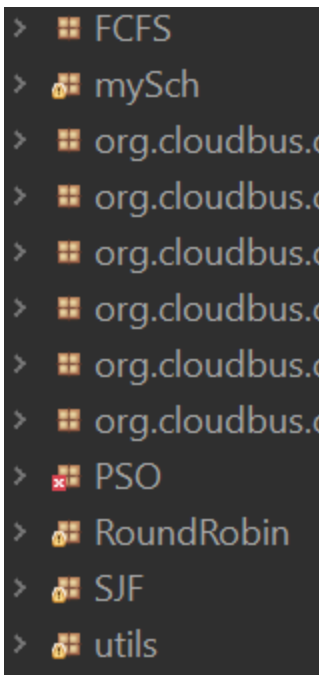
<https://github.com/michaelfahmy/cloudsim-task-scheduling/tree/master>

Step 4: Copy Source Code ke Project Eclipse

- Dari folder hasil extract, buka folder `src/`
- Di dalamnya terdapat beberapa package: `FCFS` , `mySch` , `PSO` , `RoundRobin` , `SJF` , `utils`
- **Copy semua folder/package tersebut** ke dalam folder `src/` project **CloudSimScheduling** di Eclipse
- Bisa juga dengan cara drag & drop ke `src` di Package Explorer Eclipse

Step 5: Refresh & Jalankan

- Klik kanan project > **Refresh** (atau tekan `F5`)
- Package-package baru akan muncul di Package Explorer



- Buka RoundRobin > RoundRobinScheduler.java
- Klik Kanan > Run As > Java Application

```
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
02           SUCCESS      04             04     936.48      00.1         936.58
01           SUCCESS      03             03    1897.32      00.1        1897.42
13           SUCCESS      05             05    1975.23      00.1        1975.33
04           SUCCESS      02             02    3960.94      00.1        3961.04
00           SUCCESS      06             06    4338.18      00.1        4338.28
03           SUCCESS      04             04    3497.03      936.58      4433.62
08           SUCCESS      03             03    2960.12      1897.42     4857.54
14           SUCCESS      05             05    3152.86      1975.33     5128.2
05           SUCCESS      02             02    1324.97      3961.04     5286.01
09           SUCCESS      03             03    1160.89      4857.54     6018.42
07           SUCCESS      06             06    2765.53      4338.28     7103.8
19           SUCCESS      05             05    2656.8       5128.2      7785
06           SUCCESS      02             02    2843.98      5286.01     8130
25           SUCCESS      03             03    2113.95      6018.42     8132.38
11           SUCCESS      04             04    4413.7       4433.62     8847.31
12           SUCCESS      06             06    2621.23      7103.8      9725.03
10           SUCCESS      02             02    1933.34      8130       10063.34
15           SUCCESS      02             02    646.76       10063.34   10710.1
26           SUCCESS      03             03    2781.52      8132.38   10913.9
17           SUCCESS      04             04    2484.29      8847.31   11331.6
28           SUCCESS      02             02    1369.2       10710.1   12079.3
16           SUCCESS      06             06    2360.29      9725.03   12085.32
18           SUCCESS      04             04    909.94       11331.6   12241.54
29           SUCCESS      02             02    1451.02      12079.3   13530.31
22           SUCCESS      04             04    2113.06      12241.54   14354.6
20           SUCCESS      06             06    2385.22      12085.32   14470.54
21           SUCCESS      06             06    1774.67      14470.54   16245.22
24           SUCCESS      06             06    181.38       16245.22   16426.6
23           SUCCESS      04             04    2255.32      14354.6   16609.92
27           SUCCESS      06             06    1064.74      16426.6   17491.34

Makespan using RR: 4629.656014755759
RoundRobin.RoundRobinScheduler finished!
```